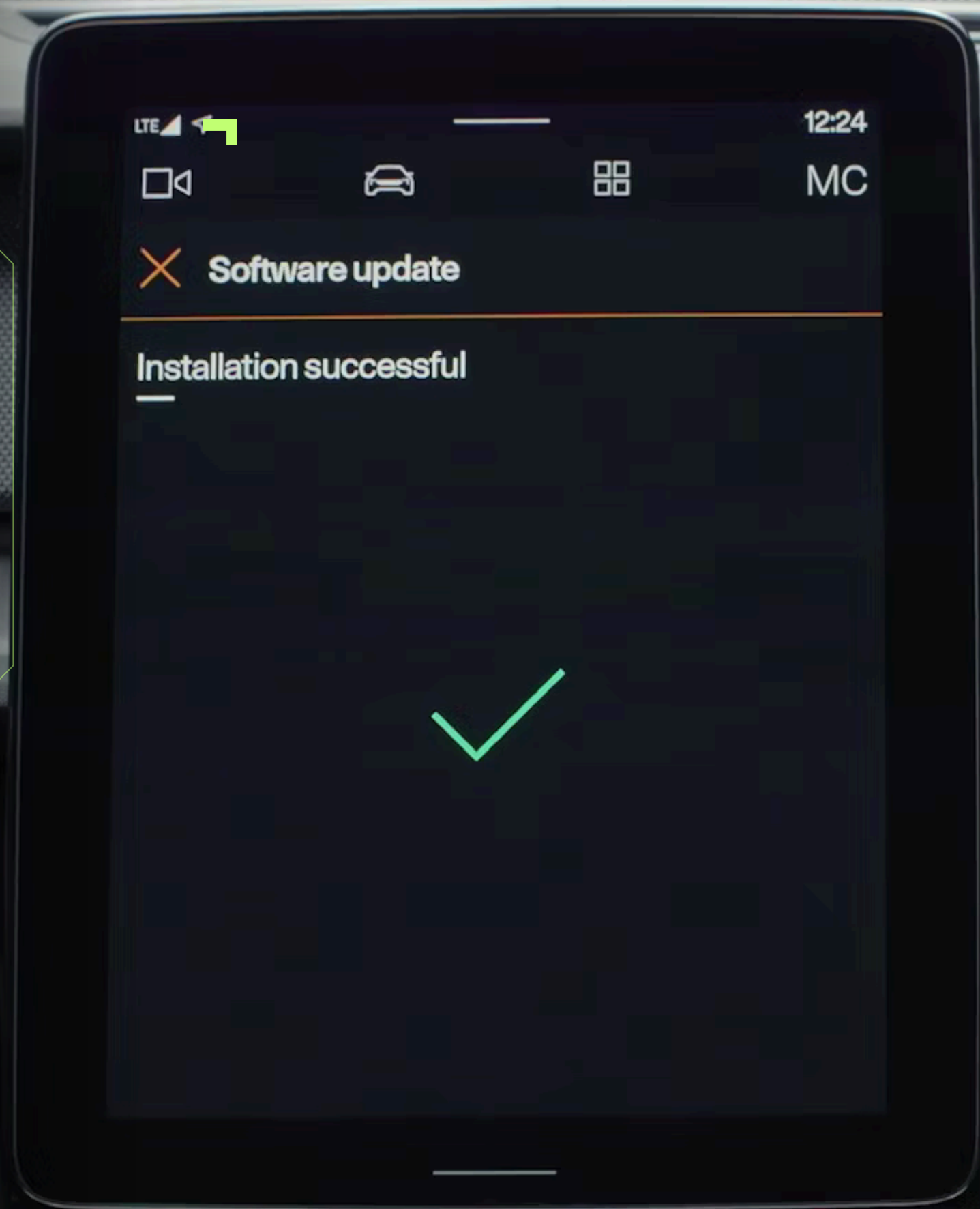


AUTOMOTIVE OTA



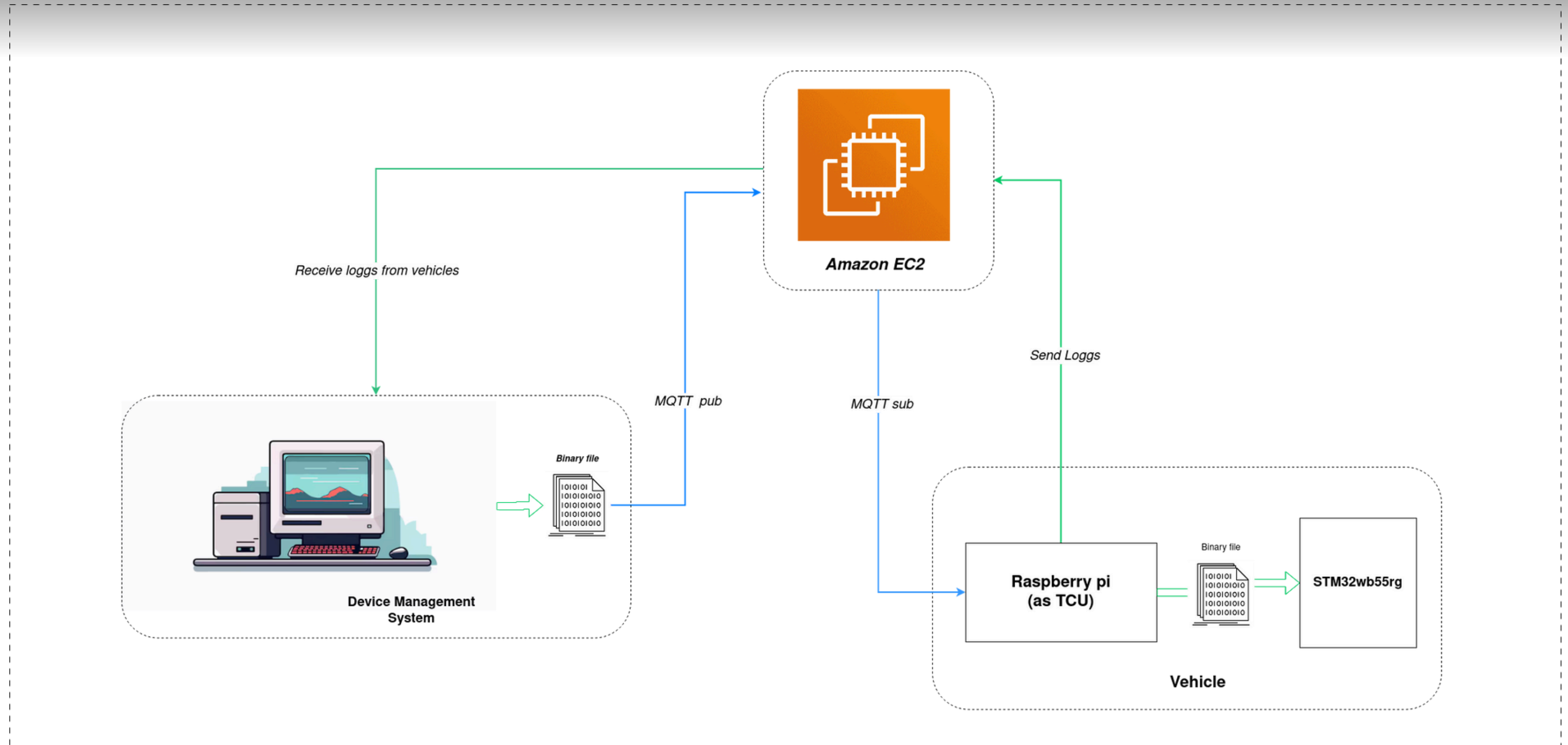
PERSONAL PROJECT



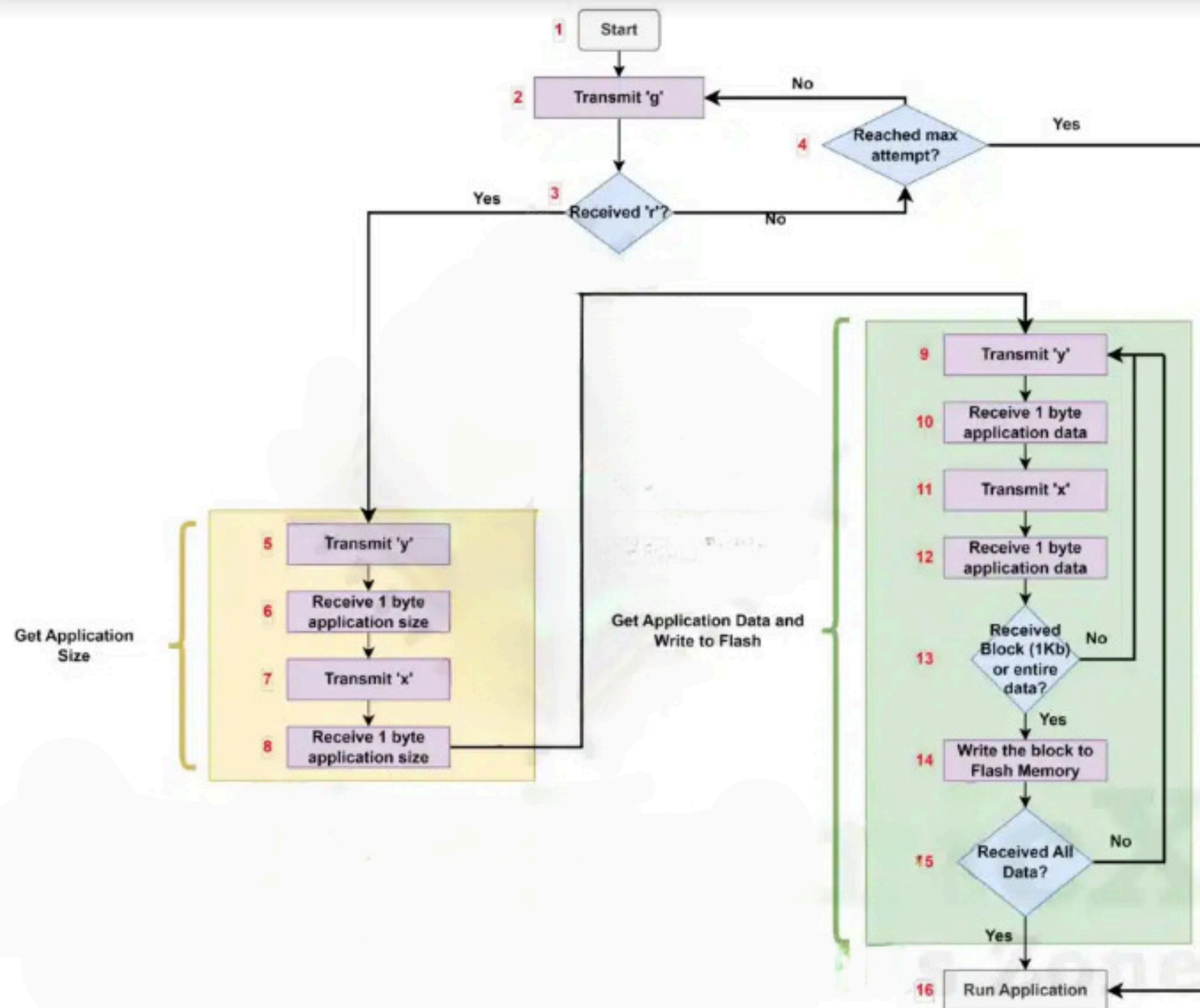
07-11-2025

MOHAMED ELHALOUA

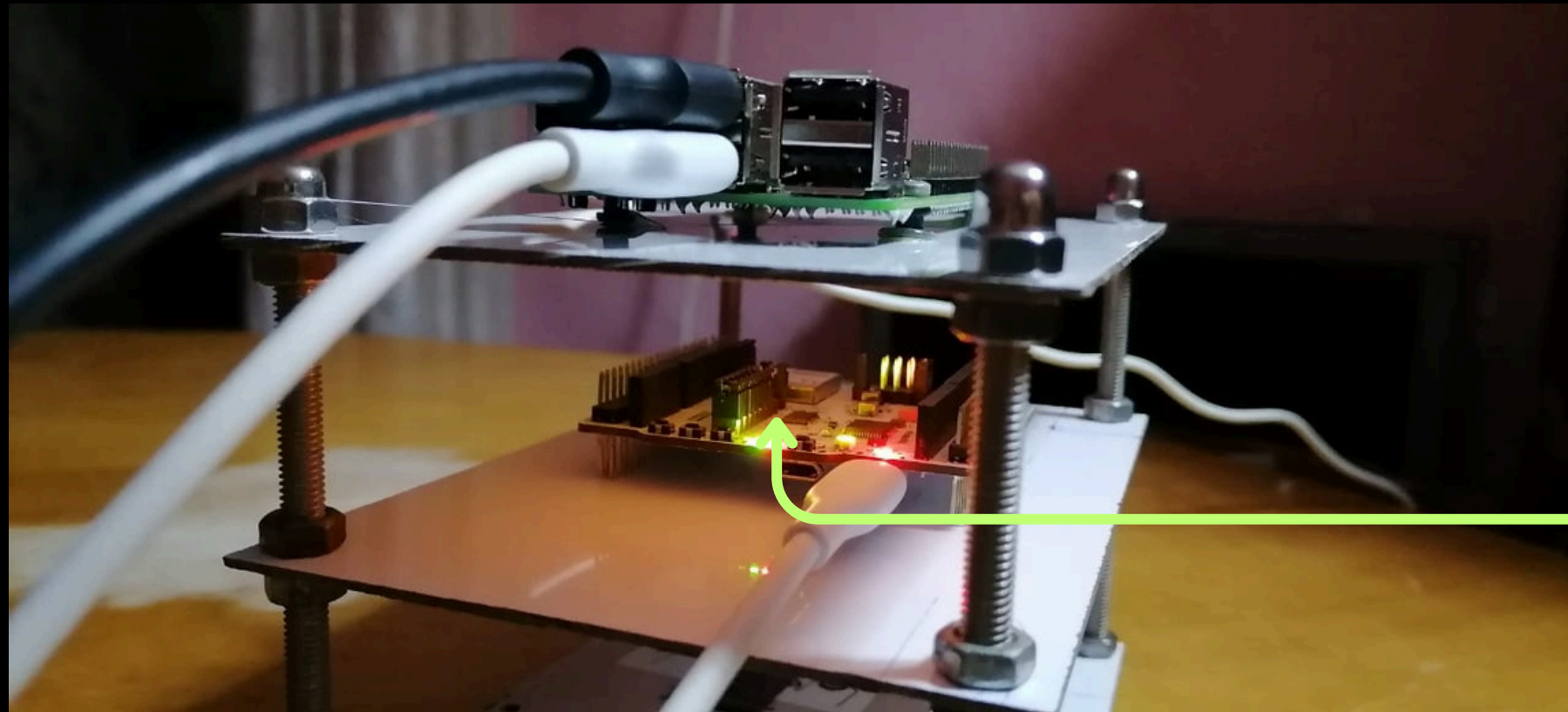
GENERAL ARCHITECTURE



UPDATE WORKFLOW

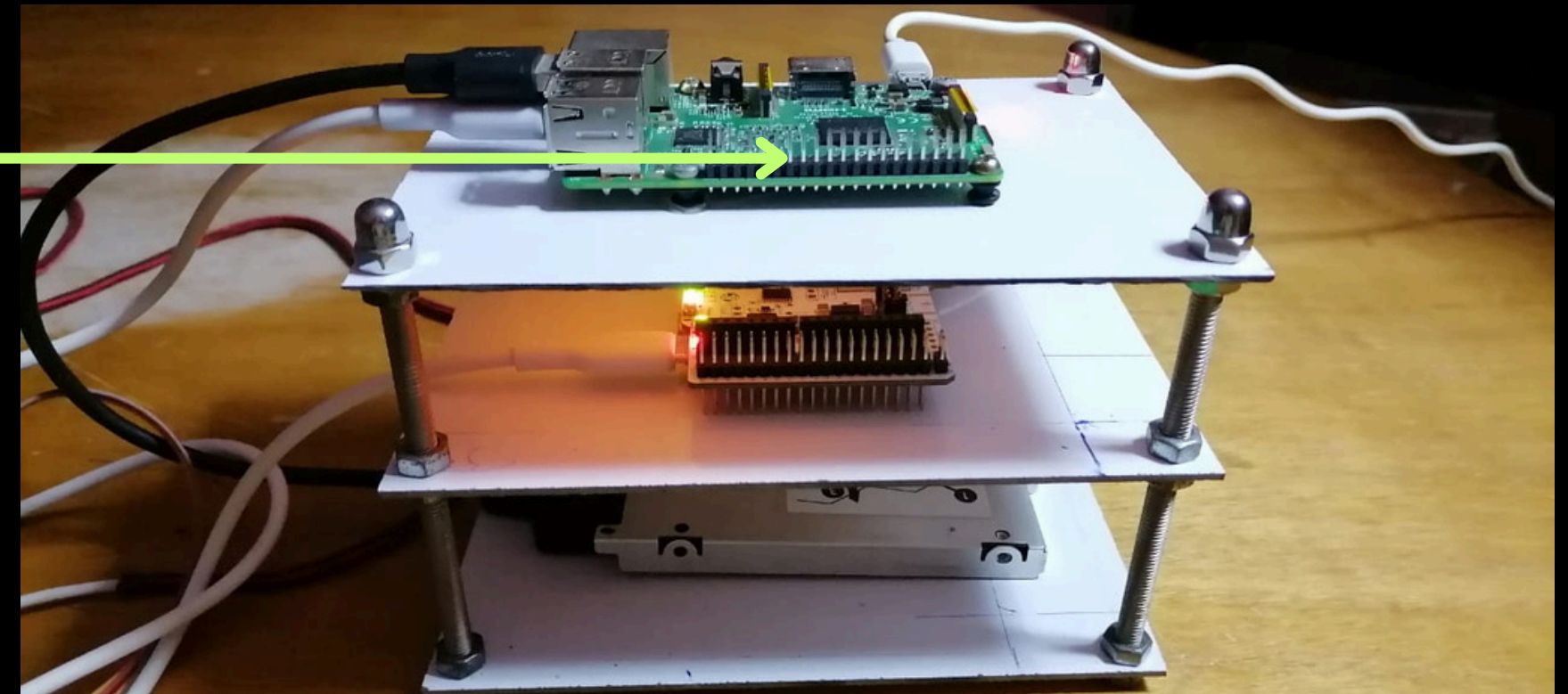


▶ PROTOTYPE SETUP



STM32WB55RG

Raspberry pi 3



RESULTS

SEND THE UPDATE FROM THE DEVICE MANAGEMENT SYSTEM TO THE VEHICLES VIA A BROKER ON THE CLOUD

```
elhaloua@192:~/STM32CubeIDE/workspace_1.19.0/09_2025_CUSTOM_BL/OTA_EV_update/OTA_EV_Sender/build$ ./OTA_EV_Send
Firmware sent successfully!
elhaloua@192:~/STM32CubeIDE/workspace_1.19.0/09_2025_CUSTOM_BL/OTA_EV_update/OTA_EV_Sender/build$
```

Send the encoded update the file to the broker on AWS 2CE using mqtt

Receive the new firmware from the broker on raspberry pi.

```
(greencrow@greencrow)-[~]
$ mosquitto_sub -h 16.170.237.131 -p 1883 -t "ota/firmware" -C 1 > /tmp/recieved_firmware.b64

(greencrow@greencrow)-[~]
$ ls /tmp/recieved_firmware.b64
/tmp/recieved_firmware.b64

(greencrow@greencrow)-[~]
$ cat /tmp/recieved_firmware.b64
Salaamo Alaykom
AAADIJULAQgpcQEIMQkBCDkJAQhBCQEISQkBCAAAAAAAAAAAAAAAAAAAAAAAAABRCQEIXwkBCAAAAABt
CQEIewkBC00LAQjtcWEI7QsBC00LAQjtcWEI7QsBC00LAQjtcWEI7QsBC00LAQjtcWEI7QsBC00L
AQjtcWEI7QsBC00LAQjtcWEI7QsBC00LAQjtcWEI7QsBC00LAQjtcWEI7QsBC00LAQjtcWEI7QsB
C00LAQjtcWEI7QsBC00LAQjtcWEI7QsBC00LAQjtcWEI7QsBC00LAQjtcWEI7QsBC00LAQjtcWEI
7QsBC00LAQjtcWEI7QsBC00LAQjtcWEI7QsBC00LAQjtcWEI7QsBC00LAQjtcWEI7QsBC00LAQjtc
WEI7QsBC00LAQjtcWEI7QsBC00LAQjtcWEI7QsBCAAAAAAQtQVMI3gzuQRLE7EESK/zAIABiyNw
```

TRANSFER THE UPDATE VIA UART

```
elhaloua@192:~/STM32CubeIDE/workspace_1.19.0/09_2025_CUSTOM_BL/UART_update_sender/PcTool$ ./etx_ota_app 25 nucleo_wb55_application.bin
Opening Port 25...
Opening Binary file : nucleo_wb55_application.bin
File size = 19916
Waiting for OTA Start
Sending r...
Sending FW Size...
Sending Data...
Block 1 Transmitted...
Block 2 Transmitted...
Block 3 Transmitted...
Block 4 Transmitted...
Block 5 Transmitted...
Final Block 5 Transmitted...
elhaloua@192:~/STM32CubeIDE/workspace_1.19.0/09_2025_CUSTOM_BL/UART_update_sender/PcTool$
```

Once the raspberry pi receives the binary file from the broker it transfer it to the stm32 via uart

START THE CUSTOM BOOTLOADER



⌘ WHEN NO VALID APPLICATION IS FOUND, AND NO DATA RECEIVED FROM THE HOST.

```
Starting custom Bootloader(2.0)
4
3
2
1
0
#####No Data Received for Firmware Update
No valid application found, staying in bootloader
```

⌘ WHEN A VALID APPLICATION IS FOUND

```
Custom Starting Bootloader(0.1)
Gonna Jump to Application
Starting Application(0.1)

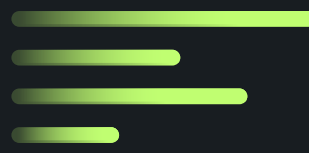
Salaam from the App.
Salaam from the App.
Salaam from the App.
Salaam from the App.
Salaam from the App.
Salaam from the App.
Salaam from the App.
Salaam from the App.
Salaam from the App.
Salaam from the App.
Salaam from the App.
Salaam from the App.
```

...

THE CUSTOM BOOTLOADER RUNNING

Once the stm32 is powered on the custom BL starts running, which will print a message blinking a LED indicating that is has started, before it move on to the firmware update function, where it will continuasly send a command'g' via UART to the TCU (in our case it is the RPI), if it receives 'r' from the TCU the new firmware transmission will started.

If no respense from the TCU is received the BL automaticly jumps to the old application.



THANK YOU